

Efficient Data Storage Systems

(Sistemas de Armazenamento de Dados Eficientes)

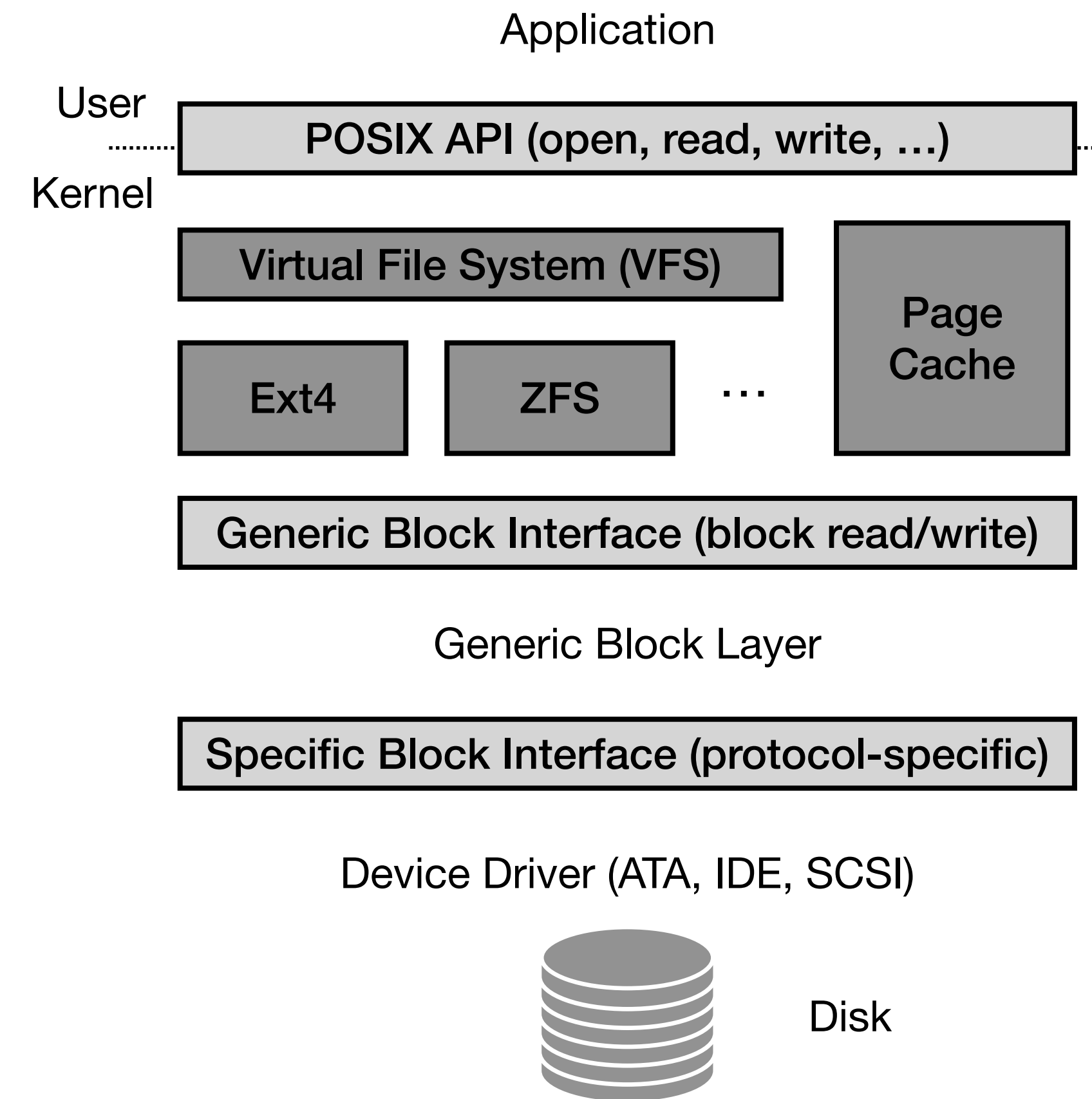
Distributed Storage Systems



Previously on SAD...

Local storage stack

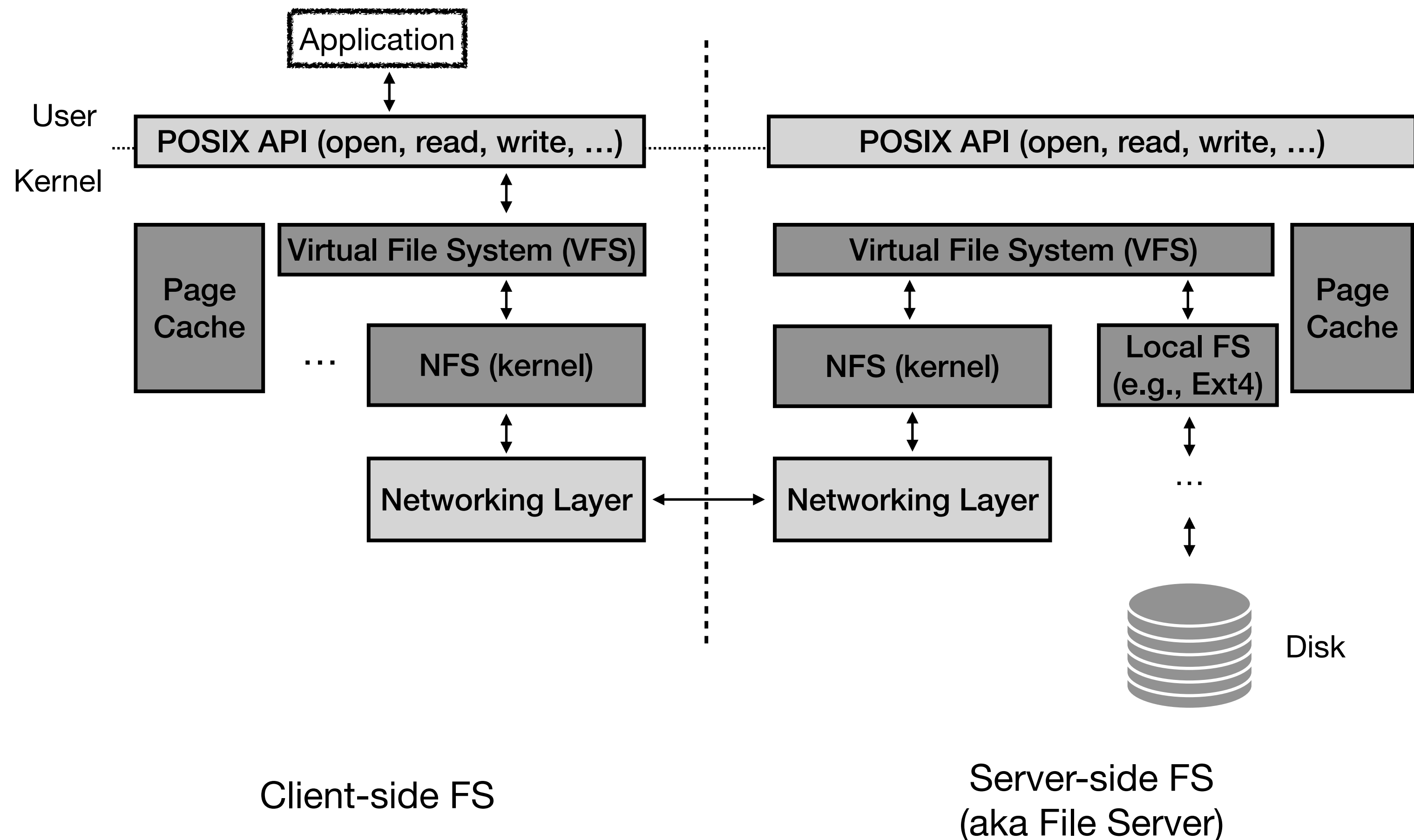
- Composed of multiple interfaces
 - Block device, file system, ...
- The **Virtual File System (VFS)** is an abstract layer
 - Specifies an interface with common operations (e.g., mounting, unmounting, accessing files, ...)
 - Facilitates the interaction with OS caches (e.g., unified page cache, inodes/directories caches)
 - Useful for applications to access **transparently** different local file systems (e.g., ext4, ZFS) and even **remote ones** (e.g., NFS)



Distributed Storage

Client-server File System (FS)

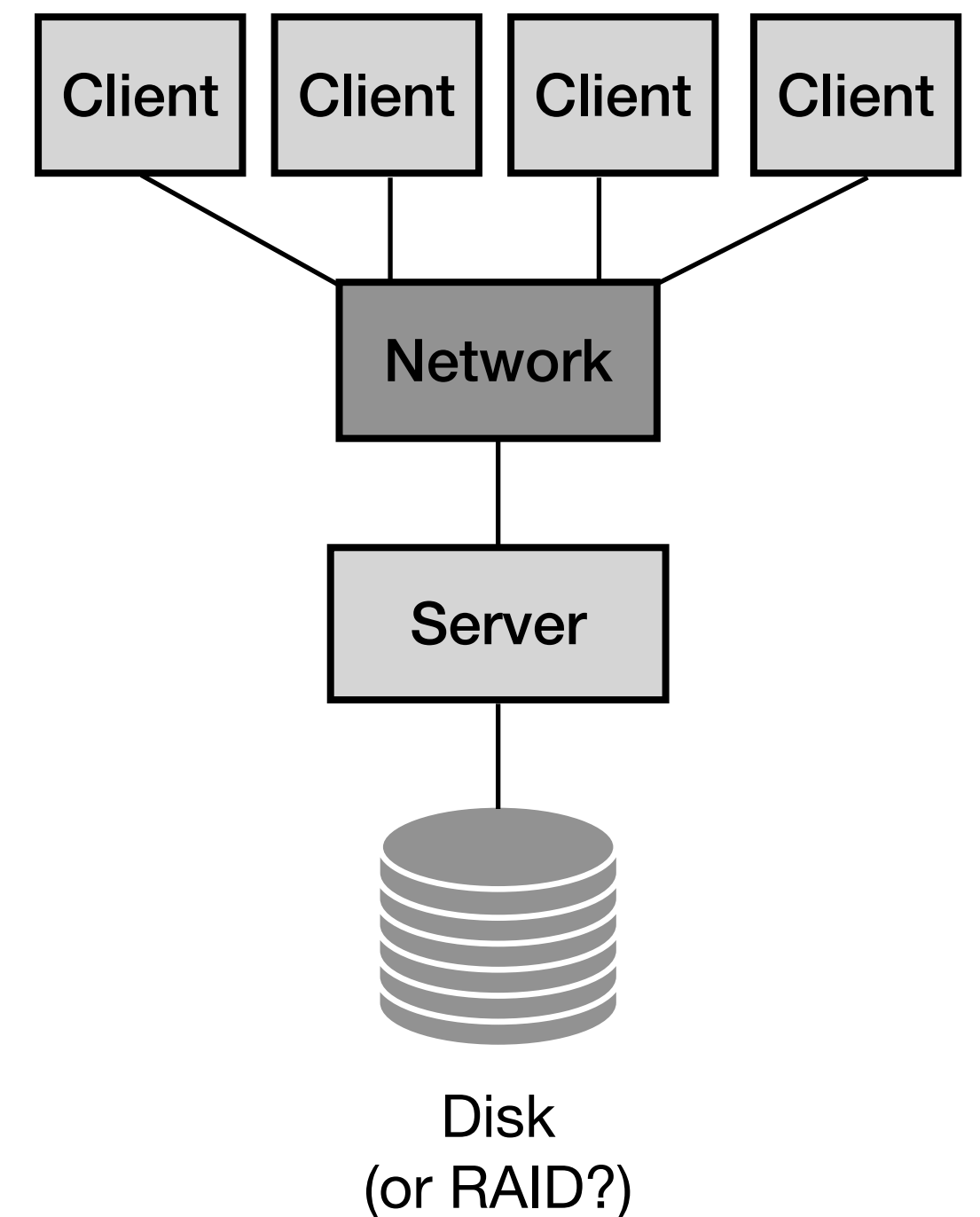
- Applications issue POSIX syscalls to the client-side FS
 - To the applications, the FS seems similar to a local one
 - **Transparency!**
- Client and server-side may do **caching** (e.g., page cache)
- A **network protocol** is used for the client and server sides to communicate (e.g., NFS)
 - **Note:** there are other storage protocols, for instance, for remote devices (e.g., iSCSI)



Network File System (NFS)

Overview

- Originally developed by Sun Microsystems in 1984
 - Sun provided an **open protocol** that specifies the message format for the client and server to communicate
 - Today, many companies sell their own NFS servers (e.g., Oracle, NetApp, EMC, IBM, ...)
- **Case study:** NFSv2 classic protocol
 - **Main Goal:** *simplicity and fast server crash recovery*
 - **Targeted environment:** *multiple-client and single-server*
 - Changes in NFSv3 (small changes) and NFSv4 (large changes, e.g., introduces a stateful approach)



Network File System (NFS)

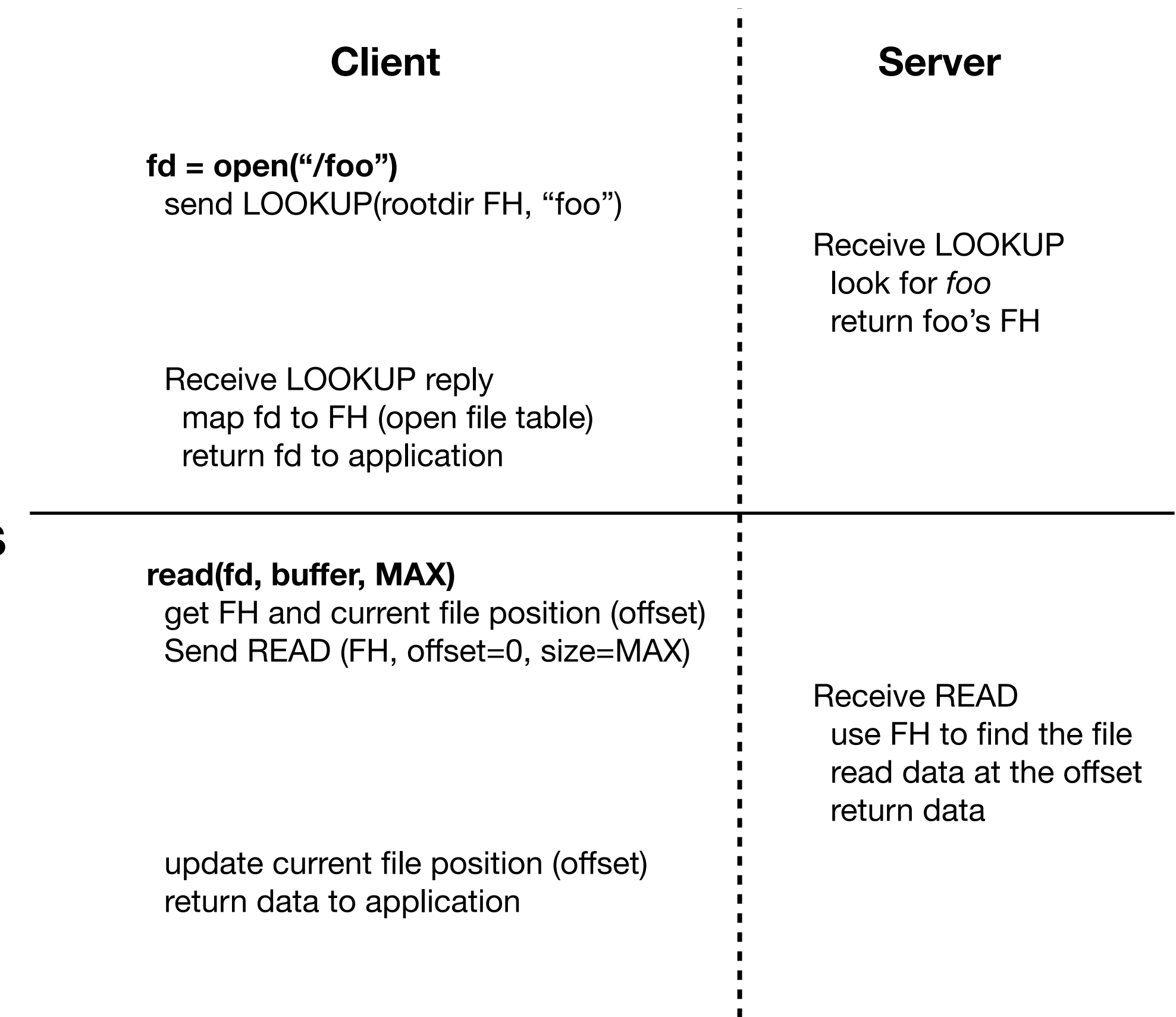
Statelessness

- ◎ **Stateless** protocol: the server does not track the client's activity (e.g., which files are opened, cached, or file offsets at the client(s))
 - Each client request provides all the needed information to do the I/O operation
 - If the server crashes, it just runs again (the client may need to retry the request)
- ◎ The opposite approach, i.e., having a **shared state** between client and server, complicates crash recovery and requires **recovery protocols**
 - *E.g.*, if clients and the server maintain a **shared state for file descriptors**
 - When the server crashes, its (ephemeral) association between descriptors and files is lost
 - When the client crashes, the server needs to know it to close opened descriptors

Network File System (NFS)

Protocol

- A **File Handle (FH)** describes a file or directory uniquely
 - Composed by a *volume identifier*, *inode number*, and *generation number* (needed when reusing inodes)
- The **LOOKUP** protocol message is used to obtain an *FH*
 - The *directory FH* and the *filename* are used as arguments
- With the *FH*, clients may issue **READ** and **WRITE** messages
 - These messages specify *size* and *offset* arguments
- The **GETATTR** message gets attributes for a given *FH* (e.g., size, modification/access times, ...)
- Other messages
 - **CREATE** (create file), **REMOVE** (remove file), **MKDIR** (create directory), **RMDIR** (remove directory)

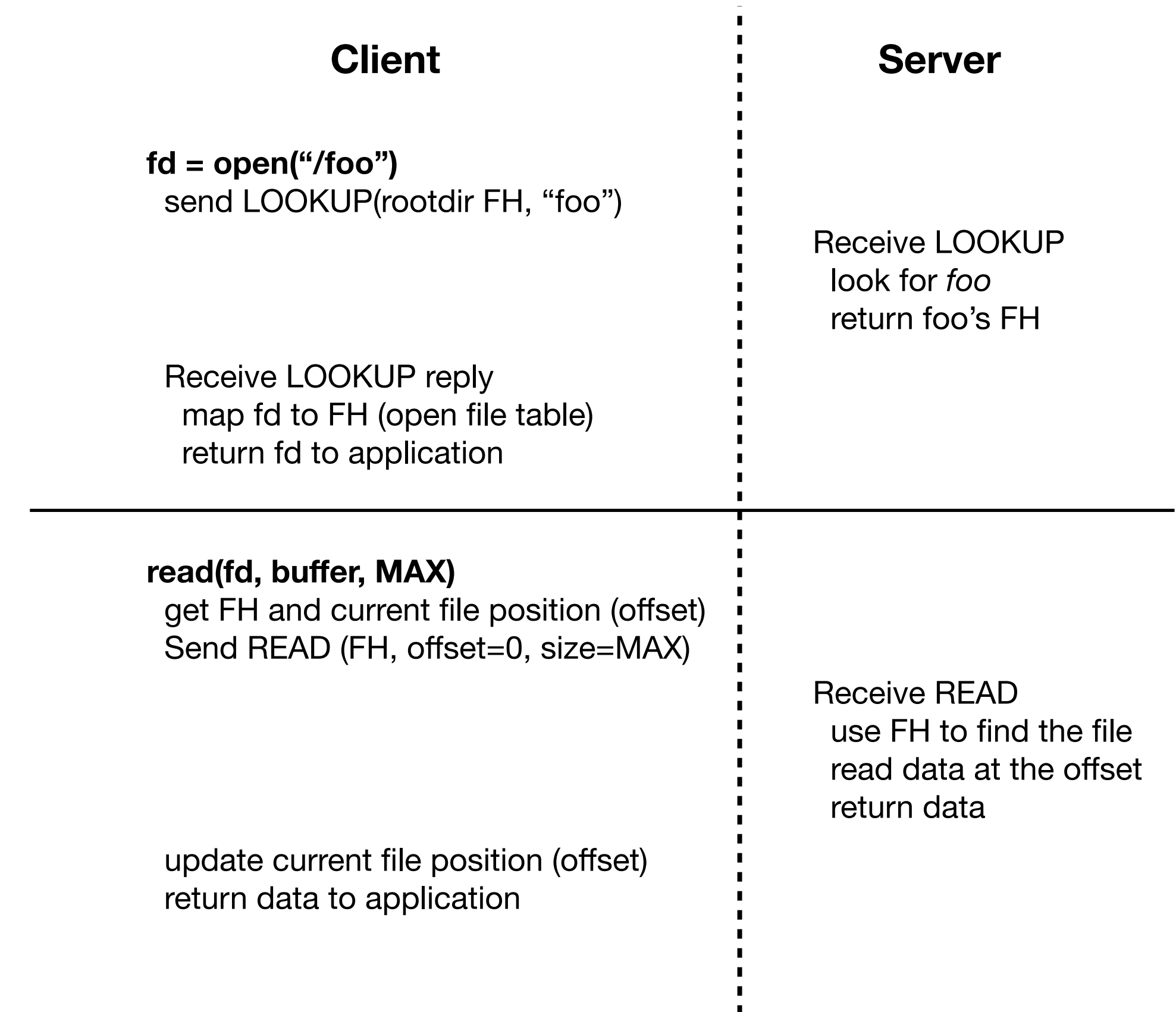


Example for file *foo* stored at the rootdir

Network File System (NFS)

Crash recovery

- The client tracks the relevant **state** for the file access
- In case of failure (e.g., request/reply message dropped, server failed), requests are **retried** by the client within a given **timeout**
- Most NFS requests are **idempotent!**
 - Performing the operation multiple times is equivalent to the effect of performing it once (e.g., *LOOKUP*, *READ*, *WRITE*)
 - *Example*: if the server persists data from a *WRITE*, and the response is not delivered, retrying the request (with the same offset and size) is idempotent
- **Careful:** Not all NFS operations are idempotent!
 - MKDIR, CREATE, REMOVE, RMDIR
 - *Example*: A retry may get an error, as a file already exists



Example for file *foo* stored at the rootdir

Network File System (NFS)

Cache consistency

- ◎ The NFS client **caches** (in-memory) data (and metadata) read from the server
 - File's blocks (not necessarily the whole file) and attributes
 - NFS cache also serves as a **write buffer**
- ◎ **Update visibility:** when do file updates become fully visible on the server?
 - **Flush-on-close policy:** When the file is closed, the NFS client ensures that the **remaining** dirty data (of that file) is flushed to the server
 - Bad, performance-wise, for temporary files (i.e., deleted shortly after creation)
Optimization: Keep short-lived files in memory and avoid flushing them

Network File System (NFS)

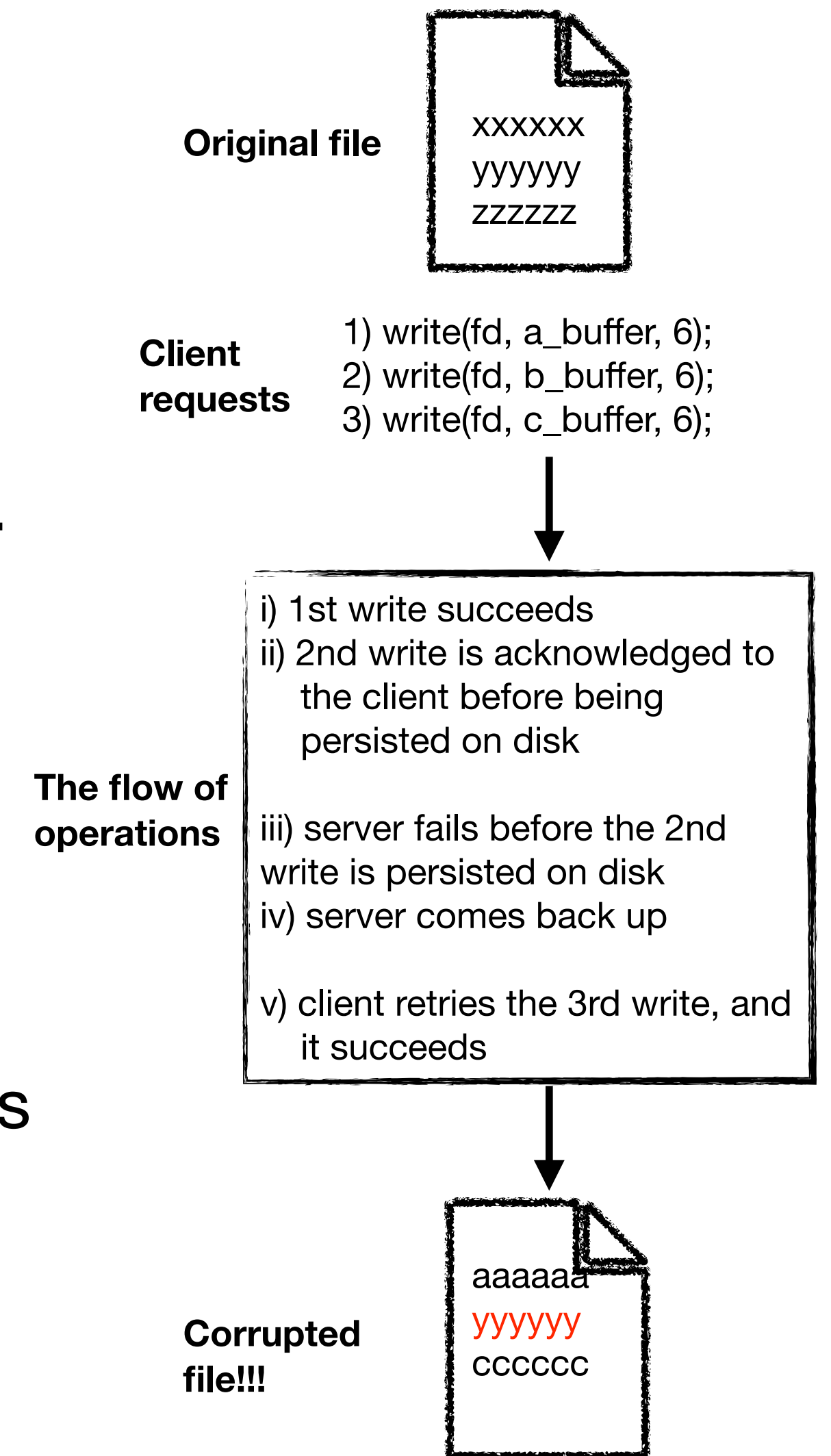
Cache consistency

● **Stale cache:** when do clients refresh their caches to see new file versions on the server?

- ▶ The NFS client checks the server before each cache access
- ▶ **Problem:** the NFS server gets overloaded with GETATTR messages...
- ▶ **Solution:** have an **attribute cache** at the client that is refreshed periodically (e.g., every 30 seconds)
Not ideal - there is a time window for cache inconsistency

● **Data durability:** The NFS server must **commit written data** to the disk before replying to the client

- ▶ Files may become corrupted with old and new blocks if the server fails
- ▶ Because of this, write performance in NFS can be a major issue...



Andrew File System (AFS)

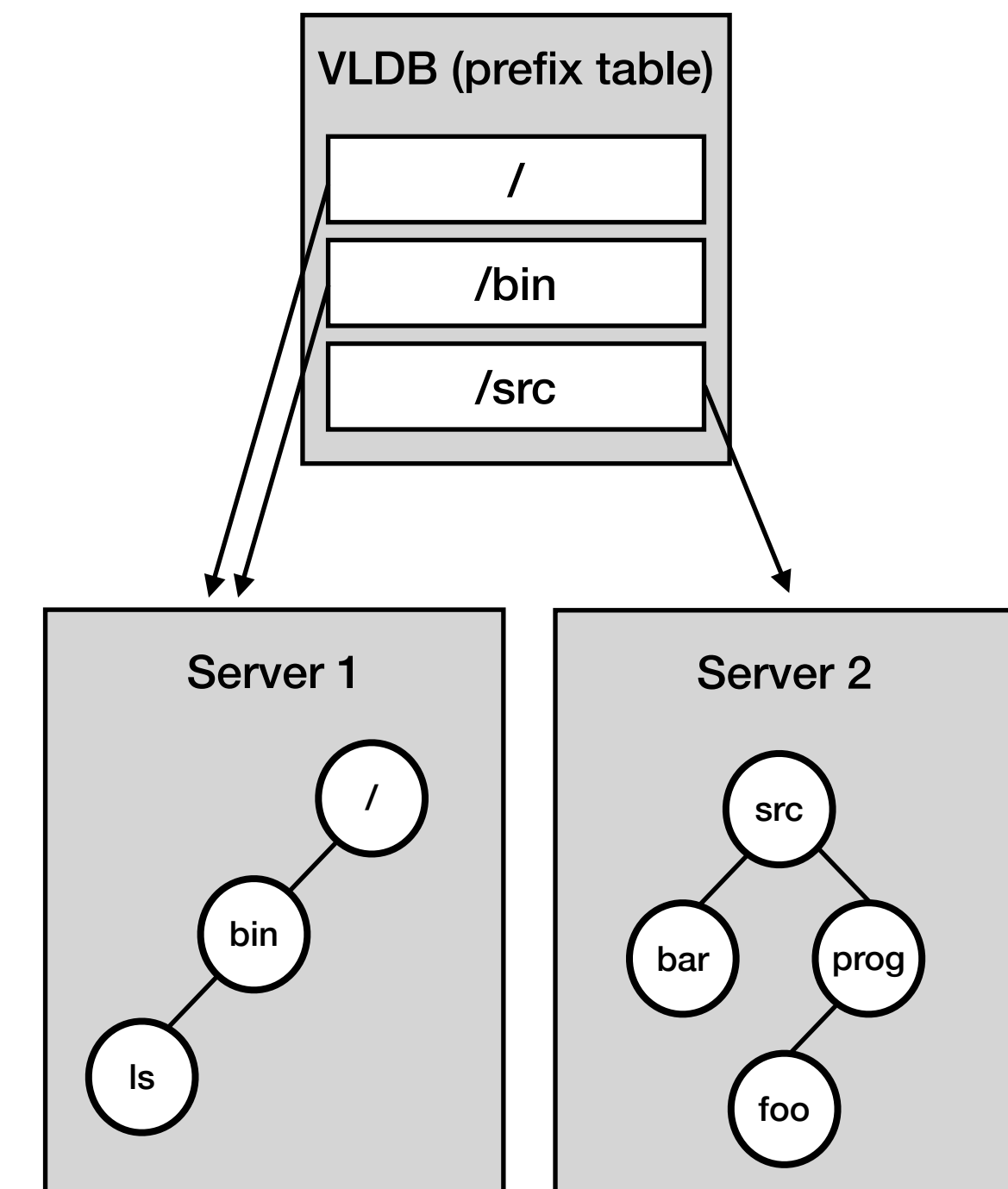
Overview

- Introduced in 1980 by Professor M. Satyanarayanan of Carnegie-Mellon University
- **Main goal:** How can one design a distributed file system such that a server can support (**scale** to) as many clients as possible?
- NFS has some problems
 - Clients must check periodically with the server to avoid stale caches
 - Cache consistency in NFS is hard to describe (depends on timeouts)

Andrew File System (AFS)

AFSv1

- AFS provides a **global metadata namespace** (unlike NFS)
i.e., all clients see the same tree of directories and files
 - Each server holds a namespace sub-tree
- A prefix table (Volume Location Database - VLDB), replicated across all servers, contains parts of namespaces located on other servers
 - Clients can cache this information to avoid extra network lookups
- The full pathname (used as a file identifier) is traversed (by the server) to access a file



Each server has a replica of the VLDB, and a metadata sub-tree colocated with the corresponding files' data

Andrew File System (AFS)

AFSv1

- ◎ **Whole-file caching** at the **local disk**¹ of each client
 - *open()* fetches the file's content from the server and stores it on the client's local disk¹
 - *read()* and *write()* requests are done to the local file copy
 - On *close()*, the full file's content is flushed to the server
- ◎ When a file is accessed (again), the client checks if the file changed at the server
 - If not, the client uses the local file copy, avoiding extra network communication

¹ In contrast, NFS caches blocks of files in memory

Andrew File System (AFS)

AFSv2

- ◎ **Path transversal cost:** with multiple clients, the server spends significant CPU time transversing directories to find a given file (the identifier is the full pathname)
 - A **file identifier (FID)** (similar to a file handle in NFS) was introduced
 - Instead of sending full pathnames to the server, the client walks the pathname, one piece at a time
 - The *directory FID* and *filename* are passed as client arguments, as in NFS
 - FID is cached at the client, potentially avoiding server lookups
- ◎ **Cache validation:** Like in the NFS protocol, the server spends considerable time validating if files cached at the clients have been updated
 - A callback is used to let the client know when a file or directory is changed!
 - The server registers a callback every time a client fetches a given file or directory

Andrew File System (AFS)

Cache consistency

- ◎ **Update visibility and cache staleness:** When a file is closed and flushed to the server, all other registered clients are notified by the server through their callbacks
 - */i.e.*, subsequent *open()* requests require fetching the file's content again from the server

- ◎ What if two clients write the same file concurrently?
 - **Last writer wins:** whichever client calls *close()* last wins
 - The end result is a file that was generated in its entirety either by one client or the other¹

Client ₁	Cache	Client ₂	Cache	Server Disk
open(F)	-		-	-
write(D)	D		-	-
	D	open(F)	-	-
	D	write(C)	C	-
	D	close()	C	C
close()	D		C	D
		open(F)	D	D
		read() -> D	D	D

Last writer wins example

¹ In NFS, writes of individual blocks may be flushed out to the server as each client is updating the file, potentially leading to a final file with mixed blocks

Andrew File System (AFS)

Crash recovery

- ◎ **Client recovery:** when a client is rebooting, callbacks from the server may be lost
 - **Solution:** Upon rejoining, clients must treat all their local copies as potentially invalid and contact the server
- ◎ **Server recovery:** if callbacks are stored in memory at the server, they are lost after a server reboot
 - **Solution:** all clients must suspect about their cached files after a server rejoins the cluster
- ◎ More complex than in NFS...

Andrew File System (AFS)

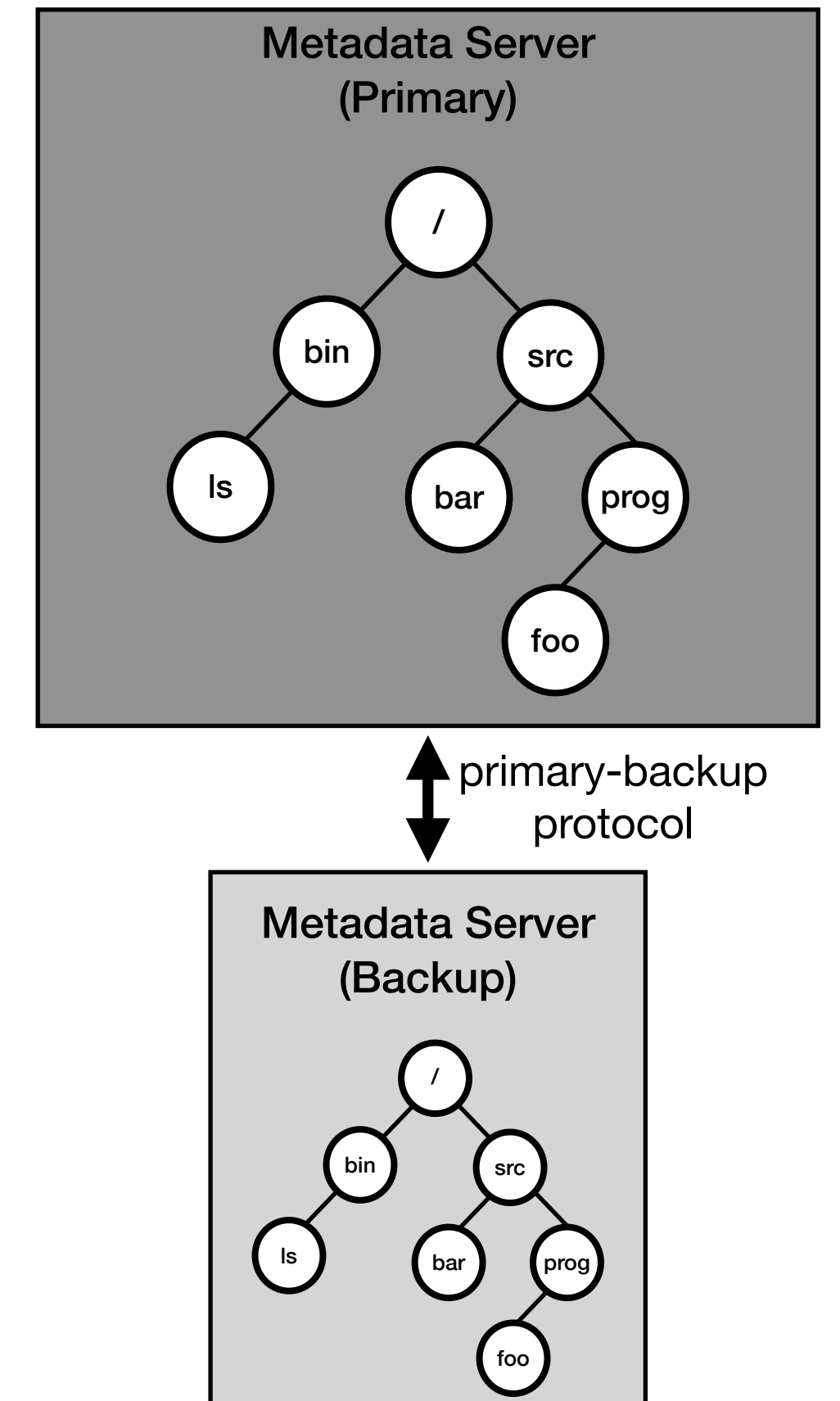
Final remarks

- AFSv1 would scale for 20 clients per server. AFSv2 improved this number to 50 clients
 - Check Section 60.7 - Remzi H. Arpaci-Dusseau, Andrea C. Arpaci-Dusseau.
Operating Systems: Three Easy Pieces, for an analysis on AFS vs NFS performance
- AFS also pioneered the inclusion of mechanisms for user authentication and access control
 - NFS had primitive support for security over several years
- While the use of AFS is now on the decline (unlike NFS), it has inspired many of the distributed file systems currently in use by the industry!

“Modern” Distributed File Systems

Namespace management

- The entire namespace is stored on a **single-node server**
 - ▶ Path translation is done in a single network round-trip
 - ▶ The single server may be a performance bottleneck (i.e., only works for small clusters or large file datasets)
 - ▶ HDFSv1 used to have a single metadata server¹
- A **prefix table** is replicated across all servers in the cluster
 - ▶ Path translation is done in two network round-trips
 - ▶ Better load balancing (sub-tree requests)
 - ▶ Moving metadata across servers is complex and costly
 - ▶ Approach used by AFS (see slide 10)



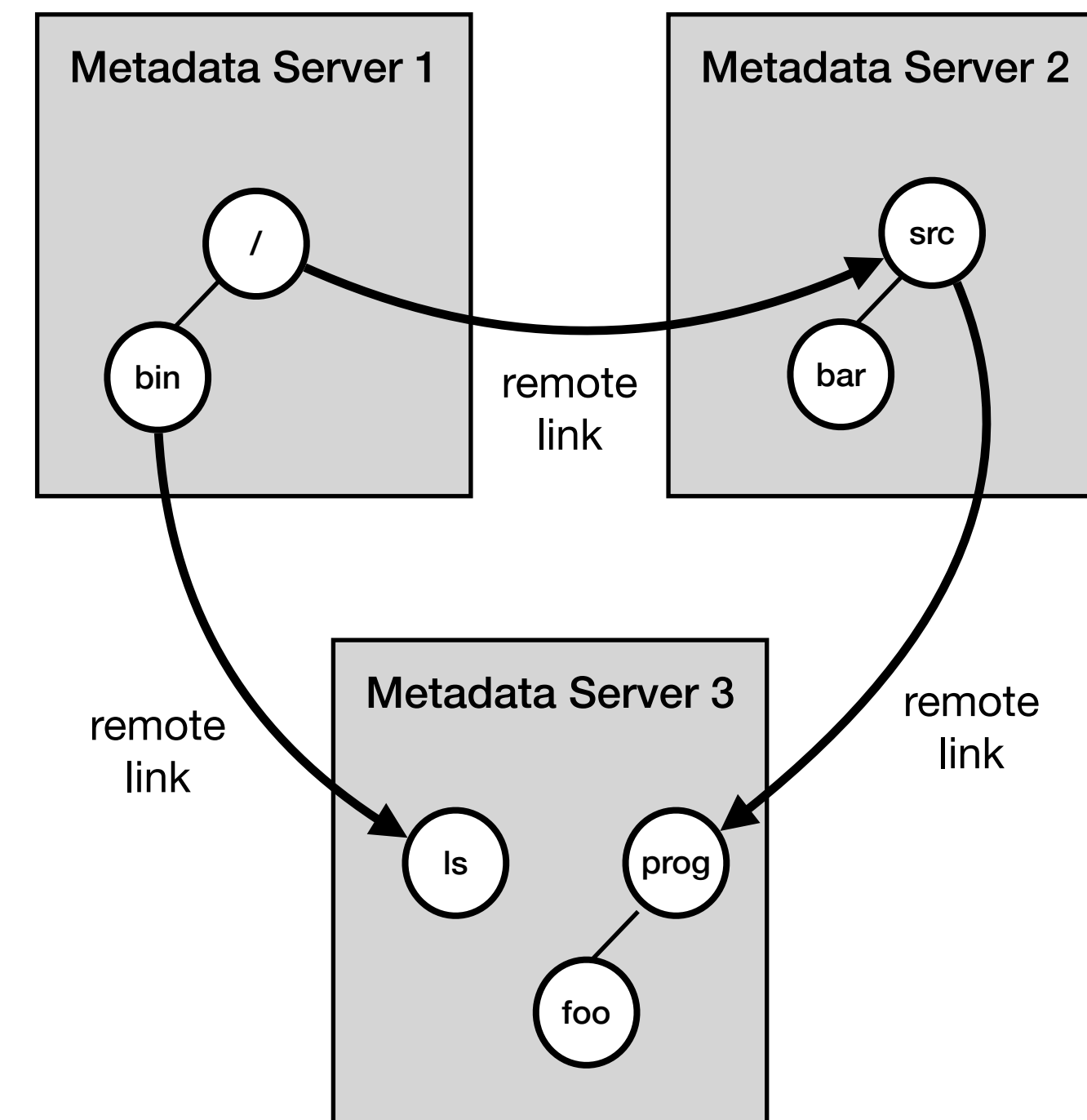
Single-node metadata server with a primary-backup strategy

¹The server can have backup (passive) replicas for fault tolerance. Version 2.0 of HDFS supports multiple servers for managing independent namespaces

“Modern” Distributed File Systems

Namespace management

- Elements of the namespace can be placed across any node and have **remote links** to the corresponding subtrees
 - Transversing a path may result in several network round-trips (clients must cache translations)
 - Even better load balancing (*i.e.*, if metadata placement is done right)
 - Migrating metadata requires creating a new remote link
- Examples
 - **Lustre Distributed Namespace** (DNE) allows splitting the namespace across metadata servers (needs admin/user intervention)
 - **Ceph’s namespace** is dynamically rebalanced and distributed across the metadata servers according to the observed workload

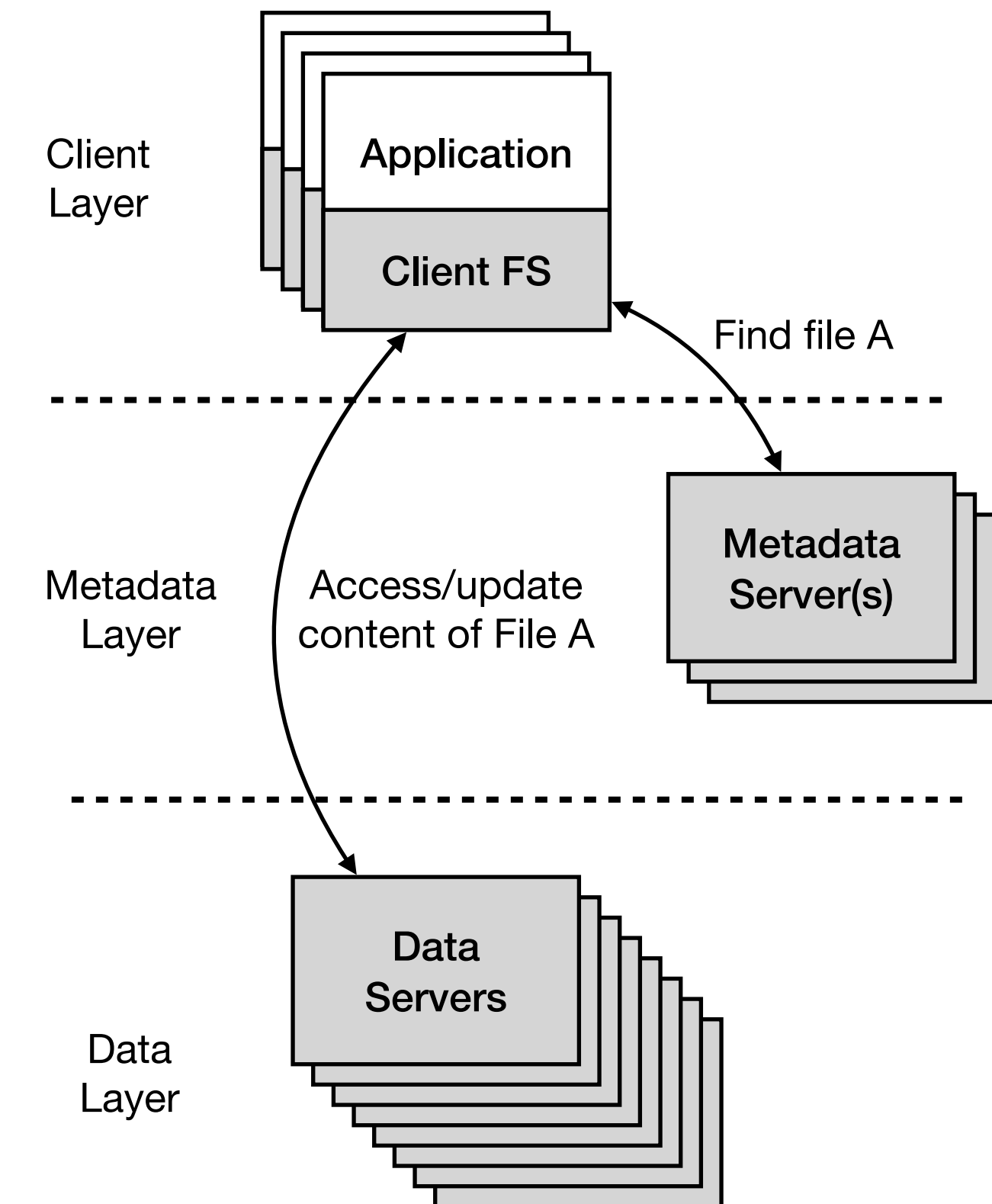


Remote link strategy

“Modern” Distributed File Systems

Data and metadata management

- Most modern file systems **separate** data and metadata management (layers) across different sets of servers
 - First, metadata servers are contacted (by the client) to get the location of data¹ (e.g., data servers holding the blocks from a given file)
 - Then, data is read/written (by the client) directly through the data servers
- **Advantages**
 - The metadata layer can be scaled independently of the data layer
 - Data can be “easily” striped (sharded) across multiple data nodes, enabling high access throughput (important for parallel file systems like Lustre)
- *Examples:*
 - NameNode and DataNodes (HDFS)
 - MDS and OSS (Lustre)
 - MDS and OSD (CephFS)



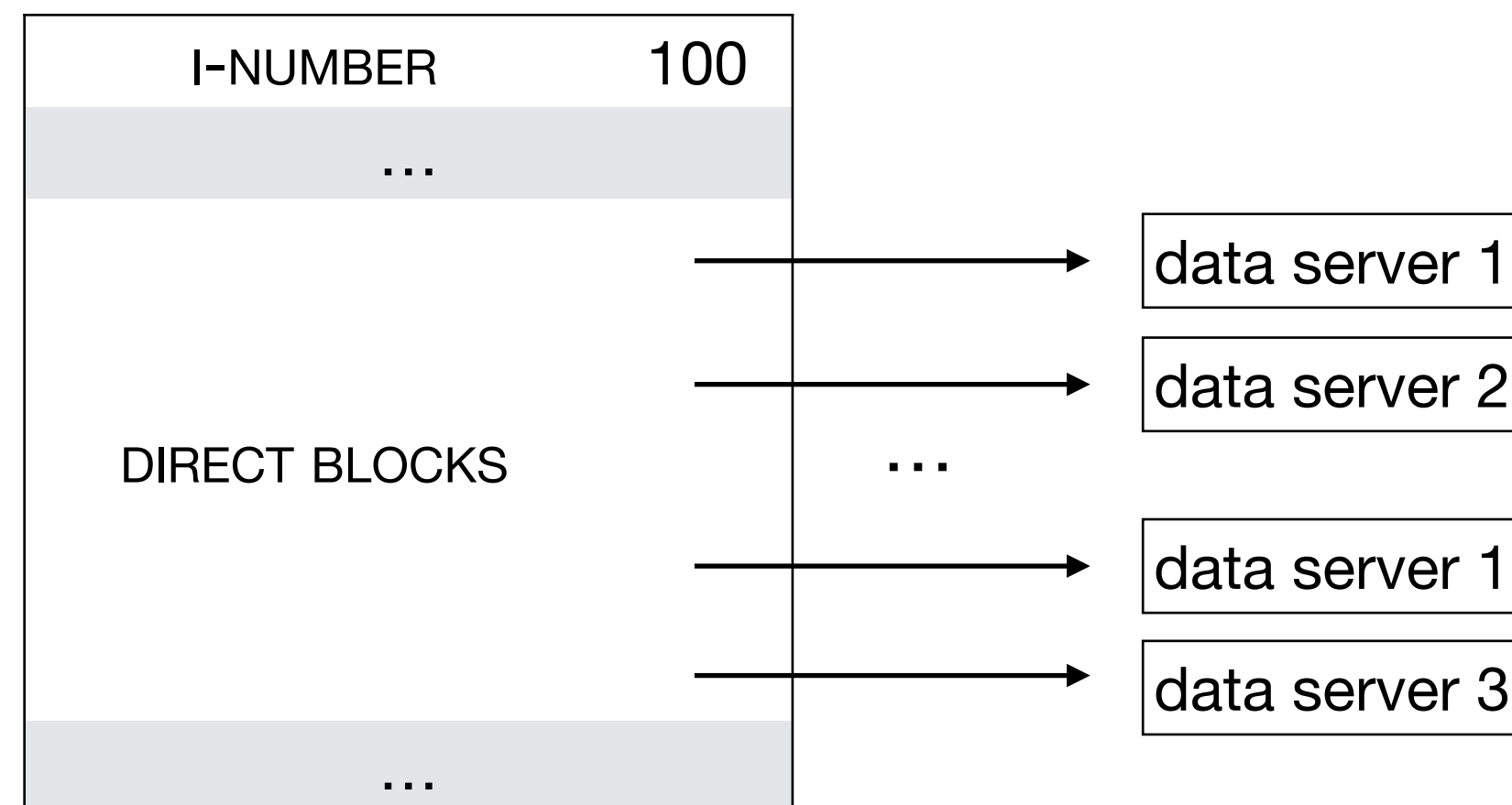
Data and metadata separation

¹ Metadata servers are also used to check for access permissions, authentication, ...

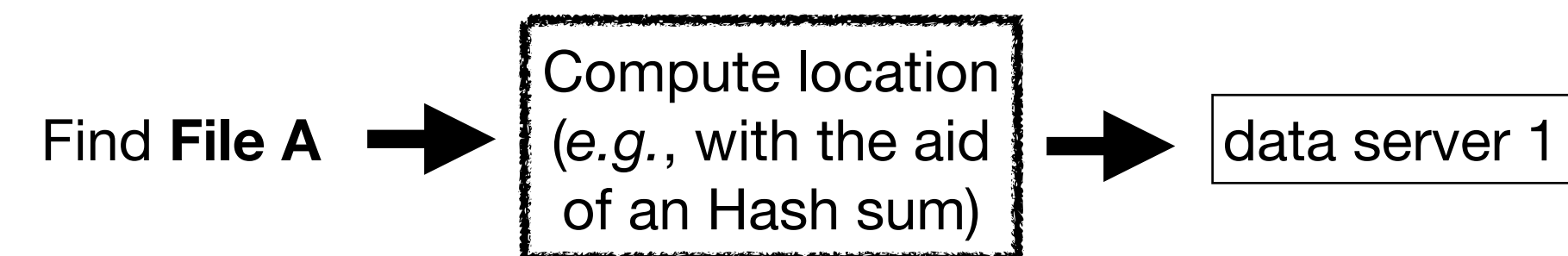
“Modern” Distributed File Systems

Data and metadata management

- ◎ Metadata servers can map the location of data with different approaches
 - **Explicitly:** The metadata inodes (kept at the metadata servers) store the location of the corresponding data blocks (e.g., HDFS and Lustre)
 - **Computed:** The location of data blocks is computed over known file attributes like the file name (e.g., the CRUSH function used in Ceph)



Simple example of **explicit** location referencing



Simple example of **computed** location referencing

“Modern” Distributed File Systems

Other final remarks

● HDFS

- Optimized for Big Data analytics, especially for the Hadoop stack
- Good for **large files** and **append-only workloads** (in-place updates are not supported natively)
- **Eventual consistency** for file creation, update, and deletion

● Ceph

- Optimized for good **performance**, **scalability**, and **fault-tolerance**
- Good for **traditional file system workloads** and for supporting **multiple interfaces** (block, file, and object)
- Adding and removing storage nodes requires rebalancing (**costly**)

● Lustre

- Optimized for **high-performance workloads** requiring high data parallelism (high throughput and low latency)
- Good for **large file workloads**
- Bad for **workloads accessing many small files** (locking and metadata servers' design)

More Information

- **Chapters 49 and 50** - Remzi H. Arpaci-Dusseau, Andrea C. Arpaci-Dusseau. **Operating Systems: Three Easy Pieces**. Arpaci-Dusseau Books, 2018.
- **Survey of Distributed File System Design Choices** - Peter Macko and Jason Hennessey. ACM TOS'22

Questions?